

SecureSign: Plataforma Web de Firma Digital y Validación Criptográfica Integrada con DevSecOps

Autor 1, Autor 2, Autor 3
Carrera de Ingeniería de Software
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador
{autor1, autor2, autor3}@espe.edu.ec

Resumen—Este trabajo presenta SecureSign, una plataforma web cliente-servidor para proteger documentos digitales mediante SHA-256, AES-256-GCM, firmas RSA-PSS y certificados X.509 emitidos por una autoridad certificadora simulada. El desarrollo integra autenticación con tokens de expiración limitada, derivación de contraseñas mediante PBKDF2, control de acceso por rol y propietario, persistencia relacional, auditoría y una canalización DevSecOps. Para relacionar las decisiones de diseño con la evidencia disponible se ejecutó una revisión rápida de literatura siguiendo las fases de Kitchenham: planificación, conducción y reporte. La evaluación experimental comprendió 140 pruebas automatizadas, análisis estático y 30 repeticiones sobre una carga de 1 081 344 bytes. Se obtuvo un tiempo medio de cifrado de 3.074 ms, descifrado de 3.008 ms y verificación de firma de 1.516 ms; las alteraciones fueron detectadas en el 100 % de las ejecuciones. Bandit y la auditoría de dependencias del frontend no reportaron hallazgos. Los resultados muestran que una arquitectura académica compacta puede integrar controles criptográficos y prácticas de seguridad continua sin sacrificar usabilidad, aunque su despliegue productivo requeriría gestión externa de secretos, TLS y protección de claves mediante hardware.

Index Terms—DevSecOps, firma digital, criptografía aplicada, AES-GCM, RSA-PSS, certificados X.509, revisión sistemática.

I. Introducción

Los documentos intercambiados por aplicaciones web requieren garantías diferenciadas: la confidencialidad limita su lectura, la integridad permite detectar cambios y la autenticidad vincula una operación con una identidad. Tratar estas propiedades únicamente al final del desarrollo contradice el principio DevSecOps de responsabilidad compartida. La literatura caracteriza DevSecOps como la incorporación continua de seguridad al flujo DevOps y destaca la automatización, el shift-left y la evaluación continua como prácticas recurrentes [1], [2], [3].

El objetivo de este estudio es diseñar, implementar y evaluar una plataforma reproducible que reúna gestión de usuarios, documentos y certificados con operaciones criptográficas. Las contribuciones son: (1) una arquitectura web funcional con controles de autorización y auditoría; (2) una síntesis de evidencia guiada por Kitchenham; y (3) resultados experimentales y de seguridad que pueden reproducirse desde el repositorio.

II. Marco teórico

II-A. Controles criptográficos

SHA-256 produce resúmenes de 256 bits y está definido por el estándar Secure Hash Standard [4]. En SecureSign se usa para identificar el contenido y comparar integridad; para contraseñas se emplea PBKDF2 con sal aleatoria y 600 000 iteraciones, pues una función rápida aislada no ofrece suficiente resistencia frente a intentos fuera de línea.

AES es el cifrador simétrico normalizado por FIPS 197 [5]. El modo GCM aporta confidencialidad y autenticación del texto cifrado, evitando separar de forma insegura el cifrado y el código de autenticación. Para firmas se aplica RSASSA-PSS con SHA-256, esquema probabilístico especificado en PKCS #1 [6]. Los certificados X.509 vinculan una clave pública con un sujeto y una entidad emisora; el prototipo valida vigencia, emisor, firma y estado de revocación.

II-B. DevSecOps

Myrbakken y Colomo-Palacios identificaron tempranamente que los controles tradicionales no acompañaban la velocidad de DevOps [1]. Rajapakse et al. sintetizaron 54 estudios y agruparon los desafíos en personas, prácticas, herramientas e infraestructura [2]. Trabajos posteriores proponen modelos que vinculan desafíos, prácticas, herramientas y métricas [7], y evidencia industrial señala la relevancia de SAST, ataques automatizados y seguridad compartida [3]. Estas conclusiones fundamentaron una canalización con pruebas, compilación, Bandit y escaneo de dependencias.

III. Metodología

III-A. Revisión rápida según Kitchenham

Se siguieron las fases de planificación, conducción y reporte propuestas por Kitchenham y Charters [8]. Se formularon dos preguntas:

- RQ1: ¿Qué prácticas y herramientas facilitan incorporar seguridad continua en una aplicación web criptográfica?
- RQ2: ¿Qué controles permiten verificar integridad, autenticidad y confidencialidad de documentos?

La búsqueda se ejecutó el 26 de julio de 2026 en índices web de IEEE Xplore, ACM DL, ScienceDirect, SpringerLink y repositorios académicos, con la cadena: (DevSecOps OR “secure software development”) AND (practice OR tool OR challenge) AND (cryptography OR “digital signature” OR certificate). Se incluyeron trabajos en inglés o español, publicados entre 2016 y 2026, con revisión por pares, texto o metadatos verificables y relación directa con las preguntas. Se excluyeron duplicados, publicidad, trabajos sin método explícito y estudios sin relación con seguridad del ciclo de vida.

La identificación produjo 14 candidatos; tras eliminar dos duplicados se revisaron 12 títulos y resúmenes. Tres se excluyeron por baja correspondencia temática y uno por falta de evidencia metodológica, quedando ocho estudios. Cada estudio recibió un punto por pregunta de investigación explícita, método descrito, evidencia trazable y discusión de limitaciones; se exigieron al menos tres de cuatro puntos. La síntesis fue narrativa debido a la heterogeneidad de métodos y métricas. El protocolo, la cadena y las decisiones se conservan para auditabilidad, tal como recomiendan las directrices [8], [9].

Cuadro I
Síntesis de la revisión de literatura

Tema	Evidencia sintetizada
Prácticas	Shift-left, automatización, seguridad compartida, evaluación continua y trazabilidad.
Herramientas	SAST, análisis de dependencias, pruebas automáticas, escaneo de contenedores e infraestructura como código.
Desafíos	Complejidad de integración, falsos positivos, cultura, competencias y balance entre velocidad y seguridad.
Métricas	Vulnerabilidades, cobertura, tiempo de operación, tasa de detección y mitigación.

III-B. Desarrollo y evaluación

El desarrollo utilizó un backlog ágil incremental. La verificación se realizó en tres niveles: pruebas unitarias de primitivas, prueba integral de API y compilación del cliente. Para el experimento se cifró, descifró, firmó y verificó una carga fija de 1 081 344 bytes durante 30 repeticiones. El tiempo se midió con reloj monotónico de alta resolución. La media se calculó como

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad (1)$$

y la detección como la proporción de firmas rechazadas después de modificar un byte. El análisis de seguridad incluyó Bandit sobre el backend y npm audit sobre dependencias de producción.

IV. Arquitectura e implementación

La interfaz React consume una API REST FastAPI. SQLAlchemy abstrae PostgreSQL en contenedores y SQLite durante pruebas locales. El sistema comprende:

- Identidad: registro, inicio de sesión, JWT HS256 con expiración, PBKDF2 y roles administrador/usuario.
- Documentos: carga restringida a 5 MB, consulta, descarga, borrado, SHA-256, firma y verificación.
- Criptografía: AES-256-GCM con nonce aleatorio, RSA-OAEP para cifrado de claves y RSA-PSS para firmas.
- PKI simulada: emisión X.509, consulta, verificación de vigencia y confianza, y revocación persistente.
- Auditoría: eventos de acceso, operaciones, fallos y validaciones sin almacenar contraseñas, claves o texto plano.

La autorización se aplica en el servidor: un usuario solo opera sobre sus registros, mientras que el administrador consulta usuarios y auditoría. Las entradas estructuradas se validan con Pydantic, los archivos se leen con límite, los nombres se neutralizan y las respuestas de error no exponen excepciones internas. La interfaz conserva la clave privada únicamente en memoria del navegador; no se persiste en el servidor.

La canalización GitHub Actions instala dependencias con versiones fijadas, ejecuta pruebas, Bandit y la compilación Vite. La literatura muestra que SAST es útil pero no completo; incluso herramientas evaluadas sobre cambios vulnerables mantienen falsos positivos y omisiones [10]. Por ello, el escaneo se complementa con pruebas funcionales y se contempla Trivy/Nmap en el laboratorio.

V. Resultados

Cuadro II
Resultados experimentales (30 repeticiones)

Métrica	Resultado
Tamaño de carga	1 081 344 bytes
Cifrado AES-GCM, media	3.074 ms
Desv. estándar del cifrado	0.767 ms
Descifrado AES-GCM, media	3.008 ms
Verificación RSA-PSS, media	1.516 ms
Detección de alteraciones	100 %
Pruebas automatizadas	140/140
Hallazgos Bandit	0
Vulnerabilidades npm audit	0

La prueba integral crea una cuenta, autentica, carga un documento, genera claves, firma, verifica, emite y valida un certificado, lo revoca y confirma su rechazo. Además comprueba que la consulta anónima de documentos devuelve HTTP 401. Las 140 pruebas pasaron; las advertencias observadas corresponden al uso interno de datetime.utcnow() en la dependencia python-jose y no a fallos funcionales.

Los resultados de la Tabla II confirman los escenarios mínimos. Un mismo contenido mantiene el resumen, cualquier modificación invalida la firma, AES-GCM recupera exactamente el texto y el certificado revocado se rechaza. Las métricas respaldan la factibilidad para archivos académicos pequeños, pero no constituyen una comparación de algoritmos ni una medición bajo carga concurrente.

VI. Verificación, validación y amenazas a la validez

La validez constructiva se fortaleció usando operaciones estandarizadas y escenarios derivados de los requisitos. Como amenaza, el cronometraje incluye efectos del sistema operativo y se ejecutó en una sola máquina; se reportan repeticiones y desviación para transparentar esa variación. La validez interna está limitada por la CA residente en proceso para la compatibilidad criptográfica, aunque la revocación funcional se persiste en base de datos. La validez externa es limitada: no se evaluaron HSM, TLS, múltiples nodos ni usuarios concurrentes.

La revisión es rápida y no pretende ser un metaanálisis. Los índices cambian y la selección fue realizada por los autores, por lo que existe riesgo de sesgo. Se mitigó mediante preguntas previas, cadena publicada, criterios explícitos y referencias con DOI o fuente normativa. La evidencia coincide en la necesidad de automatización, pero también advierte que las herramientas aisladas no sustituyen decisiones arquitectónicas ni revisión humana [2], [7], [10].

VII. Conclusiones

SecureSign satisface el alcance funcional de una plataforma académica de firma digital: integra CRUD, autenticación, cifrado, hash, firma, certificados, revocación y auditoría en una arquitectura desplegable con Docker. La evaluación detectó todas las alteraciones inducidas, mantuvo tiempos medios inferiores a 3.1 ms para la carga probada y no produjo hallazgos en los escaneos ejecutados.

La revisión guiada por Kitchenham permitió justificar la automatización y la medición como partes del diseño, no como actividades posteriores. Como trabajo futuro se propone sustituir el secreto JWT por un gestor de secretos, usar TLS mediante proxy inverso, cifrar objetos en reposo, persistir la clave de la CA en un almacén protegido, incorporar MFA, rotación de claves, límites de intentos, Alembic, DAST, Trivy y ensayos desde Kali Linux. Para valor probatorio real se requerirían sellado de tiempo, políticas de identidad verificadas y claves privadas en HSM o dispositivos del usuario.

Referencias

- [1] H. Myrbakken and R. Colomo-Palacios, “Devsecops: A multi-vocal literature review,” in *Software Process Improvement and Capability Determination*, 2017, pp. 17–29.
- [2] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, “Challenges and solutions when adopting devsecops: A systematic review,” *Information and Software Technology*, vol. 141, p. 106700, 2022.

- [3] X. Zhou et al., “Revisit security in the era of devops: An evidence-based inquiry into devsecops industry,” *IET Software*, 2023.
- [4] National Institute of Standards and Technology, “Secure hash standard (shs),” NIST, Tech. Rep. FIPS PUB 180-4, 2015.
- [5] —, “Advanced encryption standard (aes),” NIST, Tech. Rep. FIPS PUB 197-upd1, 2023.
- [6] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, “Pkcs #1: Rsa cryptography specifications version 2.2,” RFC Editor, Tech. Rep. RFC 8017, 2016.
- [7] M. A. Akbar et al., “Identifying the primary dimensions of devsecops: A multi-vocal literature review,” *Journal of Systems and Software*, vol. 214, p. 112063, 2024.
- [8] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.
- [9] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, “Systematic literature reviews in software engineering—a systematic literature review,” *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009.
- [10] S. Lee et al., “An empirical study of static analysis tools for secure code review,” *Proceedings of the ACM on Software Engineering*, pp. 691–703, 2024.